

Разбор конкурса Т-Банк

Задача 1

Ограничение времени

1 секунда

Ограничение памяти

256 МБ

Условие:

Дан массив $a_1, a_2, \dots, a_n$. Требуется определить можно ли разбить его на $n - 1$ множество таким образом, что:

- Каждый из элементов массива принадлежит ровно одному из множеств
- В каждом из множеств сумма элементов равна

Входные данные:

В первой строке записано число n ($3 \leq n \leq 10$) – длина массива

Во второй строке записано n чисел $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 100$) – элементы массива

Выходные данные:

Если массив можно разбить на $n - 1$ множество таким образом, чтобы в каждом из множеств сумма элементов была равна, выведите YES, в противном случае выведите NO.

Выводить ответ вы можете в любом регистре. Таким образом, nO, No, yEs и так далее также будут приняты.

Примечание:

В первом тесте массив можно разбить на множества $\{1, 1\}, \{2\}, \{2\}$. В нем каждый из элементов принадлежит ровно одному из $n - 1$ множеств, и сумма в каждом из множеств равна 2.

Во втором тесте можно показать, что разбиение, удовлетворяющее условию, не существует.

Примеры:

Пример 1:

Ввод:

4

1 2 1 2

Вывод:

YES

Пример 2:

Ввод:

4

6 7 6 9

Вывод:

NO

Решение

Принцип Дирихле

Если $n+1$ объект разместить в n ячейках, то хотя бы в одной ячейке окажется не менее 2 объектов

Задача сводится к простому наблюдению:

если у нас n элементов и $n-1$ групп, то по принципу Дирихле ровно в одной группе будет 2 элемента, а во всех остальных – по 1.

Значит:

- Общая сумма S должна делиться на $n-1$ – иначе точки равного разбиения быть не может
- Пусть $\text{target} = S / (n-1)$. Тогда среди элементов должна найтись ровно одна пара (i, j) , такая что $a[i] + a[j] == \text{target}$, а все остальные $n-2$ элемента сами по себе равны target

Так как $n \leq 10$, можно просто перебрать все пары индексов и для каждой проверить остаток.

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

func main() {
    reader := bufio.NewReader(os.Stdin)

    var n int
    fmt.Fscan(reader, &n)

    a := make([]int, n)
    sum := 0
    for i := range a {
        fmt.Fscan(reader, &a[i])
        sum += a[i]
    }

    groups := n - 1
    if sum%groups != 0 {
        fmt.Println("NO")
        return
    }
    target := sum / groups

    ans := "NO"
    for i := 0; i < n && ans == "NO"; i++ {
        for j := i + 1; j < n; j++ {
            if a[i]+a[j] != target {
                continue
            }
            ok := true
            for k := 0; k < n; k++ {
                if k == i || k == j {
                    continue
                }
            }
        }
    }
}
```

```
        if a[k] != target {
            ok = false
            break
        }
    }
    if ok {
        ans = "YES"
        break
    }
}

fmt.Println(ans)
}
```

Задача 2

Ограничение времени

2 секунды

Ограничение памяти

256 МБ

Условие:

Вам дан массив строк $s_1, s_2, \dots, s_n$.

Вам требуется вывести все строки, которые встречаются наибольшее количество раз в этом массиве, в лексикографическом порядке.

Строка s лексикографически меньше строки t , если и только если выполняется одно из следующего:

- s – префикс t , но $s \neq t$; или
- в первой позиции, где s и t различны, в строке s находится буква, которая встречается в алфавите раньше, чем соответствующая буква в t

Входные данные:

В первой строке записано число n ($1 \leq n \leq 2 \cdot 10^5$) – количество строк в массиве

В каждой из следующих n строк записана строка s ($1 \leq |s| \leq 10$) – очередная строка в массиве

Гарантируется, что каждая из строк содержит только строчные латинские буквы

Выходные данные:

Выведите строки, которые встречаются наибольшее количество раз в этом массиве, в лексикографическом порядке.

Примеры:

Ввод:

7

aboba

meow

abobus

abobus

aboba

abobus

aboba

Вывод:

aboba

abobus

Решение

Все просто, считаем частоты через `map[string]int`, находим максимум, собираем все строки с этим максимумом и выводим. Не забываем про быстрый ввод вывод через `bufio`

```
package main
```

```
import (
```

```

    "bufio"
    "os"
    "sort"
    "strconv"
)

func main() {
    reader := bufio.NewReaderSize(os.Stdin, 1<<20)
    writer := bufio.NewWriterSize(os.Stdout, 1<<20)
    defer writer.Flush()

    n := readInt(reader)

    counts := make(map[string]int, n)
    for i := 0; i < n; i++ {
        s := readString(reader)
        counts[s]++
    }

    maxCount := 0
    for _, c := range counts {
        if c > maxCount {
            maxCount = c
        }
    }

    result := make([]string, 0)
    for s, c := range counts {
        if c == maxCount {
            result = append(result, s)
        }
    }

    sort.Strings(result)

    for _, s := range result {
        writer.WriteString(s)
        writer.WriteByte('\n')
    }
}

func readInt(r *bufio.Reader) int {
    s := readString(r)
    n, _ := strconv.Atoi(s)
    return n
}

func readString(r *bufio.Reader) string {
    var buf []byte
    // пропускаем пробельные символы
    for {
        b, err := r.ReadByte()
        if err != nil {
            break
        }
        if b != ' ' && b != '\n' && b != '\r' && b != '\t' {

```

```

        buf = append(buf, b)
        break
    }
}
for {
    b, err := r.ReadByte()
    if err != nil {
        break
    }
    if b == ' ' || b == '\n' || b == '\r' || b == '\t' {
        break
    }
    buf = append(buf, b)
}
return string(buf)
}
pagebreak

```

Задача 3

Ограничение времени

2 секунды

Ограничение памяти

256 МБ

Кирилл скоро пойдет на пенсию, и ему хочется подготовиться к этому непростому жизненному периоду. Для начала он решил узнать, как люди развлекаются на пенсии, и сразу наткнулся на бинго.

Чтобы начать играть в эту игру, Кирилл сначала узнал правила: вам дано поле размера $n \cdot n$.

Изначально ни одна клетка не зачеркнута. Клетка в строке x и столбце y имеет номер $n \cdot (x - 1) + y$. Затем начинается сама игра, которая проходит в t раундов. В каждом из раундов вам называют номер клетки, и вы ее зачеркиваете. Если после этого хотя бы одна строка, хотя бы один столбец или одна из двух диагоналей полностью зачёркнуты, вы выигрываете.

Но условия ему показались не кристально понятными, поэтому он просит вас для конкретной партии определить, в каком раунде он впервые победит, либо же сообщить, что он не победил вообще.

Входные данные:

В первой строке записаны два числа n, t ($1 \leq n \leq 2 \cdot 10^3, 1 \leq t \leq \min(2 \cdot 10^5, n^2)$) – размер поля и количество раундов в игре

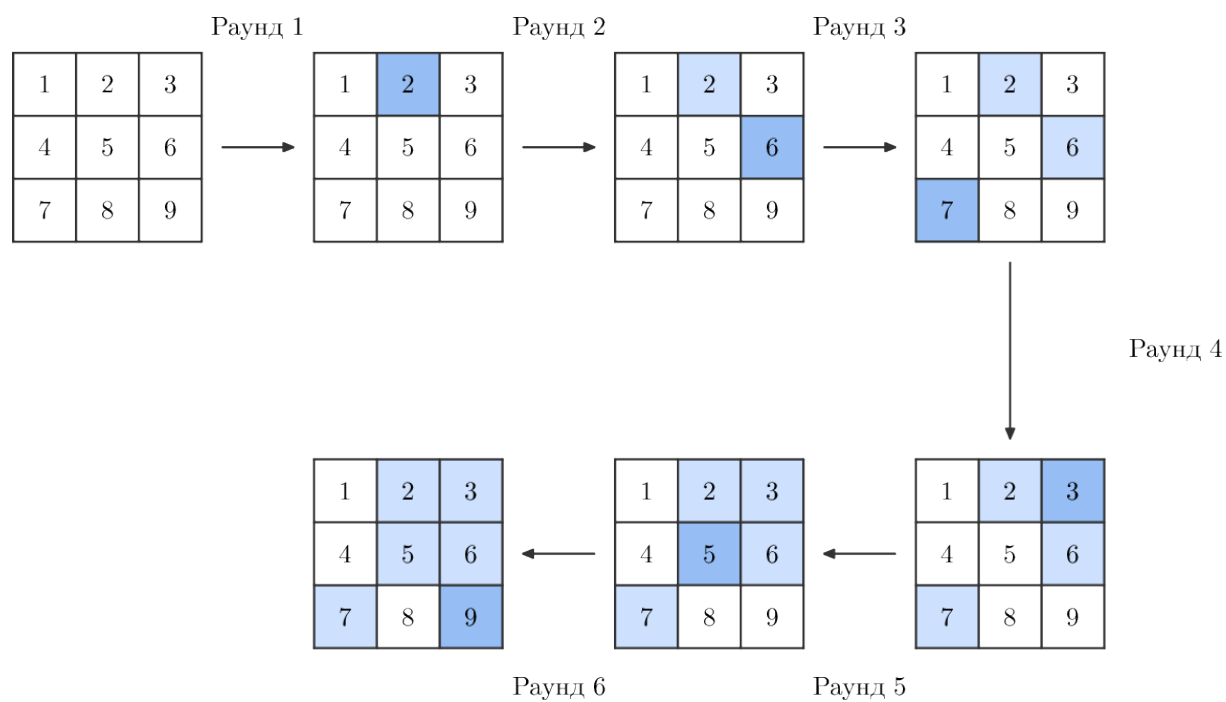
Во второй строке записаны t различных чисел $a_1, a_2, a_3, \dots, a_t$ ($1 \leq a_i \leq n^2$) – номера клеток, которые закрашивают в очередном раунде

Выходные данные:

Если Кирилл не выигрывает до конца игры, выведите -1 , в противном случае выведите номер первого раунда, в котором Кирилл победит.

Примечание

Здесь приведена игра в первом тесте. В ней после 5 раунда полностью зачеркивается диагональ.



Примеры данных:

Пример 1:

Ввод:

3 6
8 1 4 3 5 6

Вывод:

6

Пример 2:

Ввод:

3 5
2 6 7 3 5 9

Вывод:

-1

Пример 3:

Ввод:

3 6
2 6 7 3 5 9

Вывод:

5

Решение

Здесь все просто: заводим счетчики зачеркнутых клеток для каждой строки, каждого столбца и для двух диагоналей. После каждого хода увеличиваем соответствующие счетчики и проверяем, не достиг ли один из них n – если да, это и есть раунд победы

Разбор

Клетка с номером a (1-индексация) переводится в строку и столбец так:

- $x = \frac{a-1}{n}$ (0-индексированная строка)
- $y = (a - 1) \% n$ (0-индексированный столбец)

Клетка лежит на главной диагонали, если $x == y$, и на побочной, если $x + y == n - 1$.

На каждом раунде:

Увеличиваем `row[x]`, `col[y]`.

1. Если $x == y$ — увеличиваем `diag1`.
2. Если $x + y == n-1$ — увеличиваем `diag2`.
3. Если любой из этих четырёх счётчиков достиг n – это первый раунд победы, выводим номер раунда (1-индексация) и завершаем.
4. Если после всех t раундов победы не случилось – выводим -1 .

Сложность – $O(t)$, что тривиально укладывается в лимит даже при t до $2 \cdot 10^5$.

```
package main

import (
    "bufio"
    "os"
)

func main() {
    reader := bufio.NewReaderSize(os.Stdin, 1<<20)

    n := readInt(reader)
    t := readInt(reader)

    row := make([]int, n)
    col := make([]int, n)
    var diag1, diag2 int

    ans := -1
    for i := 1; i <= t; i++ {
        a := readInt(reader)
        x := (a - 1) / n
        y := (a - 1) % n

        row[x]++
        col[y]++
        if x == y {
            diag1++
        }
        if x+y == n-1 {
            diag2++
        }
    }
```

```

        if ans == -1 && (row[x] == n || col[y] == n || diag1 == n || diag2 == n) {
            ans = i
        }
    }

    writer := bufio.NewWriter(os.Stdout)
    defer writer.Flush()
    if ans == -1 {
        writer.WriteString("-1\n")
    } else {
        writer.WriteString(itoa(ans) + "\n")
    }
}

func itoa(x int) string {
    if x == 0 {
        return "0"
    }
    var buf [12]byte
    pos := len(buf)
    for x > 0 {
        pos--
        buf[pos] = byte('0' + x%10)
        x /= 10
    }
    return string(buf[pos:])
}

func readInt(r *bufio.Reader) int {
    n := 0
    c, _ := r.ReadByte()
    for c == ' ' || c == '\n' || c == '\r' || c == '\t' {
        c, _ = r.ReadByte()
    }
    for c >= '0' && c <= '9' {
        n = n*10 + int(c-'0')
        c, _ = r.ReadByte()
    }
    return n
}

```

Задача 4

Ограничение времени

2 секунды

Ограничение памяти

256 МБ

Условие:

Даны два числа n и k . Вам необходимо предоставить перестановку чисел $1, 2, \dots, n$ в которой ровно k инверсий.

Массив длины n называется перестановкой чисел $1, 2, \dots, n$ если каждое из чисел от 1 до n встречается в этом массиве ровно один раз.

Инверсией в перестановке $p_1, p_2, \dots, p_n$ называется пара индексов i, j такая, что $i < j$ и $a_i > a_j$.

Входные данные:

В единственной строке записаны два числа n, k ($2 \leq n \leq 5 \cdot 10^5, 0 \leq k \leq \frac{n(n-1)}{2}$) – длина перестановки, которую нужно предоставить, и количество инверсий в ней

Выходные данные:

Выведите n чисел – перестановку длины n , в которой ровно k инверсий. Если таких перестановок несколько, можете вывести любую из них.

Примеры данных:

Ввод:

6 4

Вывод:

1 3 2 6 5 4

Решение

Код Лемера (Lehmer code) – это особый способ кодирования каждой возможной перестановки последовательности из n чисел.

Число для кодирования каждого объекта в коде – это количество объектов, которые были доступны в этот момент (поэтому они не встречаются до позиции i), но которые меньше, чем фактически выбранный объект a_i . Это число может быть найдено прямым подсчётом инверсий или корректировкой общего количества объектов, меньших заданного, на те, которые недоступны в его положении

Идея:

Ключевой факт: для позиции i определим c_i – сколько элементов, стоящих правее позиции i , меньше a_i . Тогда суммарное число инверсий равно $\sum c_i$, причём $0 \leq c_i \leq n - i$ (сколько элементов ещё останется после выбора i -го). Максимум $\sum (n - i)$ для $i = 1..n$ как раз равен $\frac{n(n-1)}{2}$ – максимально возможному числу инверсий.

Алгоритм:

1. Идём по позициям $i = 1..n$.
2. На каждом шаге жадно берём $c_i = \min(k, n - i)$ – сколько инверсий “использовать” здесь, и уменьшаем k на c_i .

3. В качестве a_i берём $(c_i + 1)$ -ое по величине из ещё не использованных чисел

(тогда ровно c_i из оставшихся окажутся меньше него – все они справа, значит все дадут инверсию).

1. Убираем выбранное число из “доступных”.

Чтобы быстро находить “ k -ое по величине из оставшихся” и удалять – используем дерево Фенвика с бинарным подъёмом (order-statistics BIT). Каждая операция – $O(\log n)$, итого $O(n \log n)$, что комфортно укладывается в 2 секунды при $n \leq 5 \cdot 10^5$.

```
package main

import (
    "bufio"
    "os"
    "strconv"
)

var bit []int
var logN int

func update(pos, val, n int) {
    for ; pos <= n; pos += pos & (-pos) {
        bit[pos] += val
    }
}

func findKth(k, n int) int {
    pos := 0
    sum := 0
    for l := logN; l >= 0; l-- {
        next := pos + (1 << l)
        if next <= n && sum+bit[next] < k {
            pos = next
            sum += bit[next]
        }
    }
    return pos + 1
}

func main() {
    reader := bufio.NewReaderSize(os.Stdin, 1<<20)
    writer := bufio.NewWriterSize(os.Stdout, 1<<20)
    defer writer.Flush()

    n := readInt(reader)
    k := readInt64(reader)

    bit = make([]int, n+1)
    for logN = 0; (1 << (logN + 1)) <= n; logN++ {
    }

    for i := 1; i <= n; i++ {
        update(i, 1, n)
    }

    buf := make([]byte, 0, n*7)
```

```

for i := 1; i <= n; i++ {
    remaining := int64(n - i + 1)
    maxHere := remaining - 1
    c := k
    if c > maxHere {
        c = maxHere
    }
    k -= c

    val := findKth(int(c)+1, n)
    update(val, -1, n)

    if i > 1 {
        buf = append(buf, ' ')
    }
    buf = strconv.AppendInt(buf, int64(val), 10)
}
buf = append(buf, '\n')
writer.Write(buf)
}

func readInt(r *bufio.Reader) int {
    n := 0
    c, _ := r.ReadByte()
    for c == ' ' || c == '\n' || c == '\r' {
        c, _ = r.ReadByte()
    }
    for c >= '0' && c <= '9' {
        n = n*10 + int(c-'0')
        c, _ = r.ReadByte()
    }
    return n
}

func readInt64(r *bufio.Reader) int64 {
    var n int64 = 0
    c, _ := r.ReadByte()
    for c == ' ' || c == '\n' || c == '\r' {
        c, _ = r.ReadByte()
    }
    for c >= '0' && c <= '9' {
        n = n*10 + int64(c-'0')
        c, _ = r.ReadByte()
    }
    return n
}

```

Важные детали:

- k может достигать $\frac{n(n-1)}{2} = 1.25 \cdot 10^{11}$ при $n = 5 \cdot 10^5$ – это не влезает в int32, поэтому храним k в int64.
- Свой быстрый парсер ввода, потому что fmt.Fscan заметно медленнее на больших объёмах.
- Вывод собираем в один []byte буфер и пишем разом – это существенно быстрее, чем fmt.Fprintf в цикле.

Задача 5

Ограничение времени

2 секунды

Ограничение памяти

256 МБ

Условие:

Дан массив $a_1, a_2, \dots, a_n$. Рассмотрим мультимножество $S = \{a_i \cdot a_j, i < j\}$. Другими словами, S содержит все попарные произведения элементов массива.

Требуется найти k-й элемент по возрастанию в этом мультимножестве

Входные данные:

В первой строке записаны два числа n, k ($1 \leq n \leq 2 \cdot 10^5, 1 \leq k \leq \frac{n(n-1)}{2}$) – размер массива и какой элемент необходимо найти.

Во второй строке записано n чисел $a_1, a_2, \dots, a_n$ – элементы массива

Выходные данные:

Выведите k-й элемент по возрастанию в мультимножестве S.

Примечание:

В первых двух тестах отсортированное мультимножество S равно:

$[-72, -48, -18, -18, -12, -12, 0, 0, 0, 0, 0, 4, 16, 16, 54]$. В нем отмечены 4-й и 9-й элементы.

Примеры данных:

Пример 1

Ввод:

6 9
2 2 8 0 -6 -9

Вывод:

0

Пример 2

Ввод:

7 20
3 2 2 -6 -7 0 0

Вывод:

6

Пример 3

Ввод:

6 4
2 2 8 0 -6 -7 0 0

Вывод:

-18

Решение

Это классическая задача «k-й по величине элемент среди попарных произведений». Прямой перебор всех $O(n^2)$ произведений с сортировкой не пройдёт по времени при $n = 2 \cdot 10^5$ (произведений будет $= 2 \cdot 10^{10}$), поэтому нужен бинарный поиск по ответу.

Идея

Бинарным поиском ищем минимальное значение X , для которого количество пар $(i, j), i < j$, с произведением $a_i \cdot a_j \leq X$, не меньше k . Это X и будет ответом (оно обязательно совпадёт с одним из реальных произведений).

Как быстро посчитать $\text{countLE}(X)$ – число пар с произведением $\leq X$:

Сортируем массив b по возрастанию. Для каждого элемента $v = b[i]$ считаем, сколько элементов $b[j]$ (по всему массиву, включая сам себя) дают $v \cdot b[j] \leq X$:

- если $v > 0$ – произведение растёт вместе с $b[j]$, значит подходящие $b[j]$ – это префикс отсортированного массива $\rightarrow$ ищем бинпоиском границу;
- если $v < 0$ – произведение убывает при росте $b[j]$ (умножение на отрицательное меняет направление), подходящие $b[j]$ – это суффикс;
- если $v == 0$ – произведение всегда 0, подходят все n элементов, если $X \geq 0$, иначе ни одного.

Затем нужно вычесть «самого себя» – то есть случай $j = i$, который физически не является парой, но мог попасть в подсчёт, если $v \cdot v \leq X$.

Суммируем это по всем i , потом делим на 2, так как каждая неупорядоченная пара посчитана дважды (как (i, j) и (j, i)).

Это даёт $O(n \log n)$ на один вызов countLE , а всего с внешним бинпоиском – $O(n \log n \log(\text{диапазон}))$, что укладывается в лимит.

```
package main
```

```
import (
    "bufio"
    "os"
    "sort"
)

var b []int64

func countLE(x int64) int64 {
    n := len(b)
    var total int64 = 0
    for _, v := range b {
        var cnt int
        switch {
        case v > 0:
            idx := sort.Search(n, func(j int) bool { return v*b[j] > x })
            cnt = idx
        case v < 0:
            idx := sort.Search(n, func(j int) bool { return v*b[j] <= x })
            cnt = n - idx
        default:
            if x >= 0 {
```

```

        cnt = n
    } else {
        cnt = 0
    }
}
if v*v <= x {
    cnt--
}
total += int64(cnt)
}
return total / 2
}

func main() {
    reader := bufio.NewReaderSize(os.Stdin, 1<<20)

    n := readInt(reader)
    k := readInt64(reader)

    b = make([]int64, n)
    for i := 0; i < n; i++ {
        b[i] = readInt64(reader)
    }
    sort.Slice(b, func(i, j int) bool { return b[i] < b[j] })

    // границы бинарного поиска по значению X: берём максимум по модулю
    var m int64 = 0
    if abs(b[0]) > m {
        m = abs(b[0])
    }
    if abs(b[n-1]) > m {
        m = abs(b[n-1])
    }
    lo, hi := -m*m, m*m

    for lo < hi {
        mid := lo + (hi-lo)/2
        if countLE(mid) >= k {
            hi = mid
        } else {
            lo = mid + 1
        }
    }

    os.Stdout.WriteString(itoa(lo) + "\n")
}

func abs(x int64) int64 {
    if x < 0 {
        return -x
    }
    return x
}

func itoa(x int64) string {
    if x == 0 {

```

```

    return "0"
}
neg := false
if x < 0 {
    neg = true
    x = -x
}
var buf [24]byte
pos := len(buf)
for x > 0 {
    pos--
    buf[pos] = byte('0' + x%10)
    x /= 10
}
if neg {
    pos--
    buf[pos] = '-'
}
return string(buf[pos:])
}

func readInt(r *bufio.Reader) int {
    return int(readInt64(r))
}

func readInt64(r *bufio.Reader) int64 {
    var n int64 = 0
    neg := false
    c, _ := r.ReadByte()
    for c == ' ' || c == '\n' || c == '\r' || c == '\t' {
        c, _ = r.ReadByte()
    }
    if c == '-' {
        neg = true
        c, _ = r.ReadByte()
    }
    for c >= '0' && c <= '9' {
        n = n*10 + int64(c-'0')
        c, _ = r.ReadByte()
    }
    if neg {
        n = -n
    }
    return n
}

```

Задача 6

Ограничение времени

2 секунды

Ограничение памяти

256 МБ

В очередной день в Т-Банке решили устроить турнир по крикету среди всех стажёров. Всего в турнире участвуют 2^n стажёров, пронумерованных числами от 1 до 2^n (да, их очень много).

Турнир проходит по олимпийской системе. Изначально всех стажёров выстраивают в ряд в порядке их номеров. После этого проводится n раундов. В каждом раунде происходит следующее:

- Порядок оставшихся в турнире стажёров сохраняется.
- Стажёры разбиваются на пары слева направо: первый играет со вторым, третий с четвёртым и так далее.
- В каждой паре проводится матч, после которого победитель остаётся в турнире, а проигравший выбывает.

Таким образом, после каждого раунда число оставшихся участников уменьшается вдвое, а после n -го раунда остаётся ровно один стажёр – победитель турнира.

Также, чтобы турнир был интереснее для участников, организаторы подготовили Т-Мани за хорошее выступление на турнире. А точнее, если стажёр под номером i одержал ровно j побед за весь турнир, он получает $c_{i,j}$ Т-Мани. Если же стажёр ни разу не выиграл, то он ничего не получает.

Организаторов турнира не очень сильно волнуют результаты, но они хотят смотивировать участников как можно сильнее. Поэтому вам требуется сказать, какое максимальное количество Т-Мани они могут раздать, если победителя каждого из матчей они могут выбрать произвольным образом.

Входные данные:

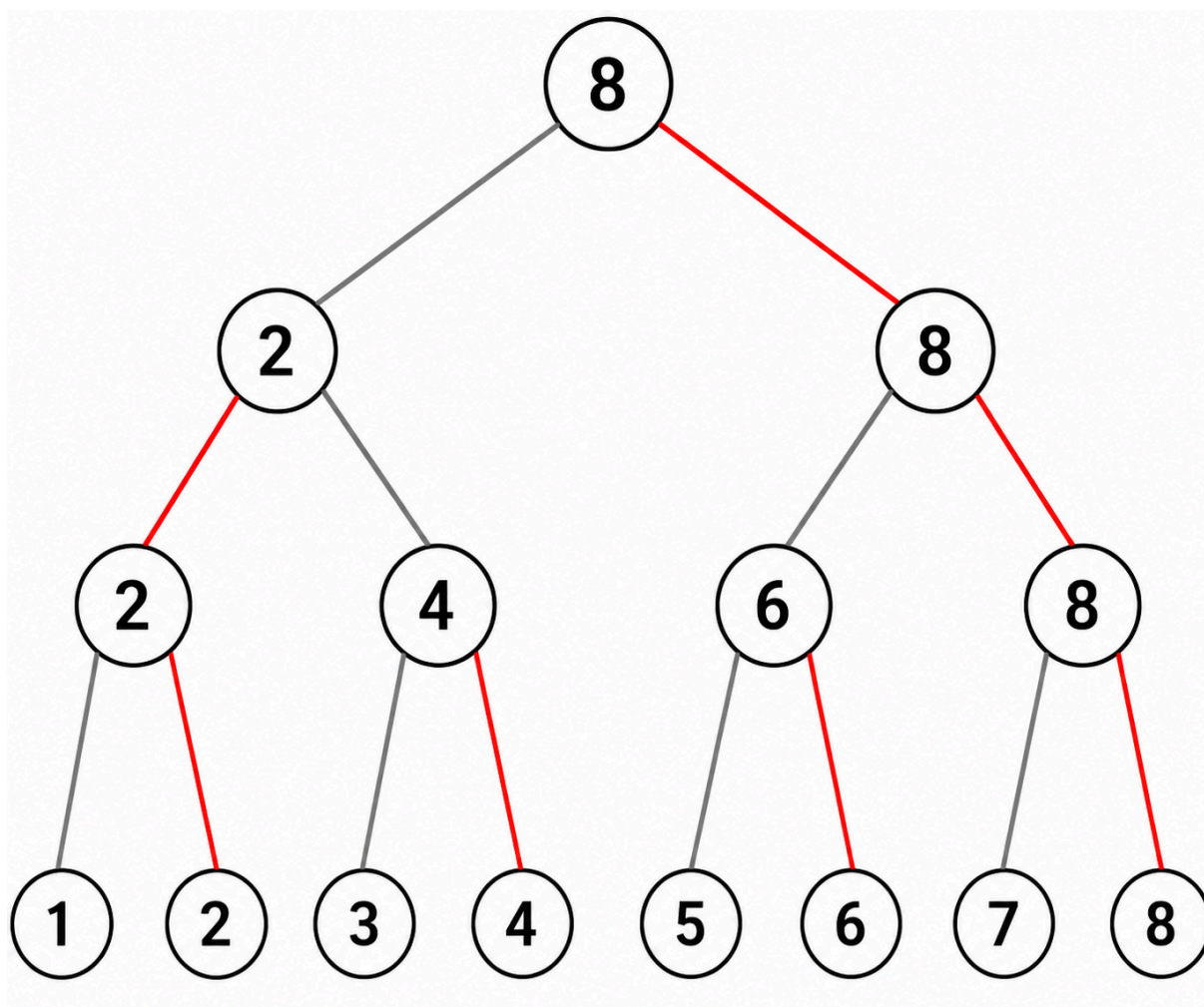
В первой строке записано единственное число n ($1 \leq n \leq 16$) – количество раундов в турнире

В следующих 2^n строках содержится описание призовых для каждого стажёра. В i -строке записаны n чисел $c_{i,1}, c_{i,2}, \dots, c_{i,n}$ ($1 \leq c_{i,j} \leq 10^9$), где $c_{i,j}$ количество Т-Мани, которое получает i -й стажёр за j побед на турнире.

Выходные данные:

Вывести единственное число – какое максимальное количество Т-Мани могут раздать организаторы

Примечание:



Здесь нарисован один из примеров оптимального турнира в первом тесте. В нём:

Игроки под номерами 1,3,5,7 ни разу не выиграли, поэтому получают 0.

Игроки под номерами 4,6 выиграли ровно один раз, поэтому они получают 4 и 5 соответственно.

Игрок под номером 2 выиграл дважды, поэтому он получает 5.

Игрок под номером 8 выиграл трижды, поэтому он получает 4.

Таким образом, суммарно организаторы выплатили 18 Т-Мани. Можно показать, что больше нельзя.

Примеры:

Ввод:

```
3
1 3 1
1 5 1
2 2 1
4 2 1
1 5 4
5 1 3
1 2 1
3 3 4
```

Вывод:

18

Решение

Ключевое наблюдение

Структура турнира – это в точности дерево merge sort: на шаге k объединяются два соседних блока размера 2^{k-1} в блок размера 2^k . Каждый участник i , дойдя до раунда k и проиграв в нём, до этого успел выиграть ровно $k - 1$ матчей – значит, его выплата фиксируется именно в момент проигрыша и равна $c[i][k - 1]$ (1-индексация по числу побед).

Отсюда идея: считать не «кто выиграл каждый матч», а сразу идти по уровням дерева и для каждого блока решать, кого из него выгоднее сделать «победителем блока» – то есть кандидатом на прохождение дальше, а всех остальных участников блока – учитывать сразу, фиксируя их выплаты.

Определение состояния

$dp[i]$ – максимальная суммарная выплата всем участникам текущего блока, кроме i , при условии, что именно i становится победителем этого блока (то есть его собственная награда за победы в этом блоке пока не учтена – она будет зафиксирована позже, когда он либо проиграет на более высоком уровне, либо станет абсолютным чемпионом).

Изначально (блоки размера 1) $dp[i] = 0$ для всех i – награды ещё не за что платить.

Переход (слияние двух блоков в один)

Пусть на шаге k сливаются левый блок L и правый блок R , каждый размера 2^{k-1} .

Если победителем объединённого блока становится участник $i \in L$, то из R кто-то должен проиграть в этом самом матче, имея на тот момент $k - 1$ побед – значит, тот, кто проиграл, получает $c[j][k - 1]$. Организаторы выбирают этого j так, чтобы максимизировать сумму:

$$\text{bestRight} = \max_{j \in R} (dp[j] + c[j][k-1])$$

Тогда для каждого $i \in L$:

$$\text{newDp}[i] = dp[i] + \text{bestRight}$$

Симметрично для победителей из R :

$$\text{bestLeft} = \max_{i \in L} (dp[i] + c[i][k-1]) \quad \text{newDp}[j] = dp[j] + \text{bestLeft} \text{ для } j \in R$$

Важно: bestLeft и bestRight считаются до обновления dp , по старым значениям – иначе получится использование ещё не подтверждённого состояния.

Смысл формулы: какой бы участник блока L ни стал победителем блока, «партнёр по поражению» из R в любом случае один и тот же (лучший по $dp[j] + c[j][k-1]$), потому что выбор соперника для конкретного матча в финале блока никак не связан с тем, кто именно из L дошёл до этой позиции — организаторы вольны выбирать исходы всех предыдущих матчей независимо.

Ответ

После n -го слияния остаётся один блок размера 2^n , и $dp[i]$ — суммарная выплата всем, кроме чемпиона i . Чемпион должен получить ещё приз за n побед:

$answer = \max \text{ over } i \text{ of } (dp[i] + c[i][n])$

(в 0-индексации массива c — $c[i][n-1]$).

Пример разбора твоего теста

Раунд 1 не приносит выплат (проигравшие имеют 0 побед). Раунд 2 фиксирует выплаты за 1 победу, раунд 3 (финал) — за 2 победы, и в самом конце к ответу добавляется приз чемпиону за 3 победы. Именно эта цепочка и даёт ответ 18.

```
package main

import (
    "bufio"
    "os"
    "strconv"
)

func main() {
    reader := bufio.NewReaderSize(os.Stdin, 1<<20)

    n := readInt(reader)
    total := 1 << n

    c := make([][]int64, total)
    for i := 0; i < total; i++ {
        c[i] = make([]int64, n)
        for j := 0; j < n; j++ {
            c[i][j] = readInt64(reader)
        }
    }

    dp := make([]int64, total)

    for k := 1; k <= n; k++ {
        blockSize := 1 << k
        half := blockSize / 2
        rewardIdx := k - 2

        for s := 0; s < total; s += blockSize {
            var bestLeft, bestRight int64 = -1 << 62, -1 << 62
            for i := s; i < s+half; i++ {
                v := dp[i]
                if rewardIdx >= 0 {
                    v += c[i][rewardIdx]
                }
                if v > bestLeft {
```

```

        bestLeft = v
    }
}
for j := s + half; j < s+blockSize; j++ {
    v := dp[j]
    if rewardIdx >= 0 {
        v += c[j][rewardIdx]
    }
    if v > bestRight {
        bestRight = v
    }
}
for i := s; i < s+half; i++ {
    dp[i] += bestRight
}
for j := s + half; j < s+blockSize; j++ {
    dp[j] += bestLeft
}
}
}

var ans int64 = -1 << 62
for i := 0; i < total; i++ {
    v := dp[i] + c[i][n-1]
    if v > ans {
        ans = v
    }
}

writer := bufio.NewWriter(os.Stdout)
defer writer.Flush()
writer.WriteString(strconv.FormatInt(ans, 10))
writer.WriteString("\n")
}

func readInt(r *bufio.Reader) int {
    return int(readInt64(r))
}

func readInt64(r *bufio.Reader) int64 {
    var n int64 = 0
    c, _ := r.ReadByte()
    for c == ' ' || c == '\n' || c == '\r' || c == '\t' {
        c, _ = r.ReadByte()
    }
    for c >= '0' && c <= '9' {
        n = n*10 + int64(c-'0')
        c, _ = r.ReadByte()
    }
    return n
}

```

Задача 7

Ограничение времени

1 секунда

Ограничение памяти

256 МБ

Условие:

Дан взвешенный неориентированный граф из n вершин и $n-1$ ребра. Гарантируется, что эти ребра образуют дерево. После этого поступает q запросов:

- В граф добавляется ребро между вершинами u и v с весом w . После этого требуется сообщить суммарный вес ребер минимального остовного дерева данного графа.

Обратите внимание, что все добавленные рёбра остаются в графе.

Минимальное остовное дерево графа – это дерево, ребра которого являются подмножеством ребер этого графа, с минимально возможной суммой весов ребер.

Входные данные:

В первой строке записаны два числа n, q ($2 \leq n \leq 2 \cdot 10^5, 1 \leq q \leq 2 \cdot 10^5$) – количество вершин в графе и количество запросов

В следующих $n - 1$ строках идет описание изначальных ребер графа. Каждое ребро задается тремя числами a, b, c ($1 \leq a < b \leq n, 1 \leq c \leq 10$) – его концами и весом соответственно.

Гарантируется, что эти ребра образуют дерево.

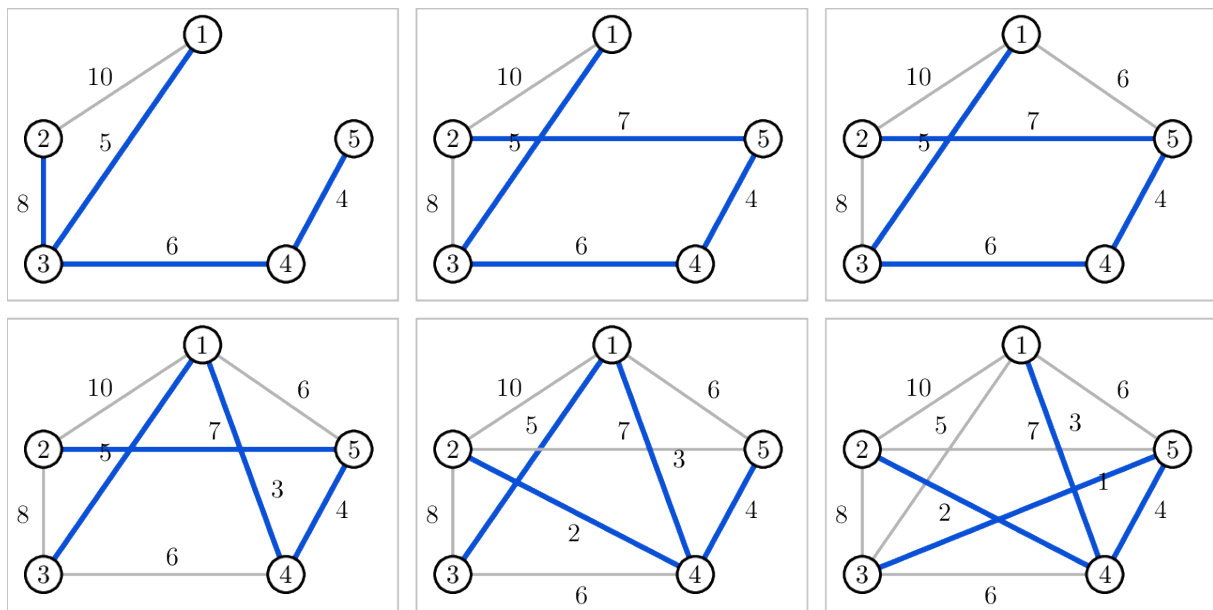
Затем в следующих q строках идет описание запросов. Каждый запрос задается тремя числами u, v, w ($1 \leq u < v \leq n, 1 \leq w \leq 10$) – концами добавляемого ребра и его весом соответственно.

Выходные данные:

Выведите q чисел – суммарный вес ребер в минимальном остовном дереве после каждого из запросов.

Примечание:

На этой картинке показан граф в первом тесте после каждого из запросов, а синим отмечены ребра, которые лежат в минимальном остовном дереве.



Примеры данных:

Ввод:

```

5 6
1 2 10
2 3 8
3 4 6
4 5 4
1 3 5
2 5 7
1 5 6
1 4 3
2 4 2
3 5 1

```

Вывод:

```

23
22
22
19
14
10

```

Решение

Изначально граф – дерево, то есть уже единственное (и минимальное) остовное дерево. При добавлении ребра (u, v, w) в связный граф образуется ровно один цикл – путь от u до v в текущем остовном дереве плюс новое ребро. Чтобы обновить MST, достаточно классического наблюдения:

Если вес w нового ребра меньше, чем вес самого тяжёлого ребра на пути $u \rightarrow v$ в текущем дереве – выгодно выбросить это самое тяжёлое ребро и вставить новое. Если w не меньше – новое ребро никогда не попадёт в MST, ничего не меняется.

Так как таких запросов до $2 \cdot 10^5$, а дерево при этом само меняется (мы физически заменяем рёбра), наивный пересчёт пути каждый раз через DFS/LCA даст $O(n)$ на запрос – слишком медленно. Нужна структура, которая умеет за $O(\log n)$:

находить максимальное по весу ребро на пути между двумя вершинами; вырезать ребро (cut) и добавлять новое (link).

Это классическая область применения Link-Cut Tree (LCT).

Приём «ребро как вершина»

LCT изначально хранит веса в вершинах, а не в рёбрах. Стандартный трюк: каждое ребро (a, b, w) представляем как отдельную “виртуальную” вершину e с весом w , и вместо ребра $a-b$ линкуем $a-e-b$. Тогда вес любого ребра – это вес соответствующей виртуальной вершины, и путь $u \rightarrow v$ в LCT всегда чередует настоящие вершины (вес -1 , заведомо меньше любого реального веса) и виртуальные (вес ребра). Максимум по пути автоматически даёт нужное ребро.

Алгоритм по шагам

1. Читаем дерево, для каждого исходного ребра создаём виртуальную вершину-ребро и линкуем её между концами. Суммируем веса – это стартовый ответ.
2. На каждый запрос (u, v, w) :
 - `makeRoot(u)`, `access(v)` – путь $u..v$ становится единой splay-структурой с корнем v ; берём `mx[v]` (максимальный вес на пути) и `mxPos[v]` (индекс этой виртуальной вершины);
 - если $w < mx[v]$ – обновляем сумму (`total += w - mx[v]`), вырезаем старое ребро (cut с обеих сторон от виртуальной вершины), создаём новую виртуальную вершину для (u, v, w) и линкуем её;
 - выводим текущую сумму.

Сложность – $O((n+q) \log n)$ амортизированно, что комфортно укладывается в 1 секунду.

```
package main
```

```
import (  
    "bufio"  
    "os"  
    "strconv"  
)
```

```
var (  
    ch    [][]int32  
    par   []int32  
    rv    []bool  
    val   []int64  
    mx    []int64  
    mxPos []int32  
    eu    []int32  
    ev    []int32  
    stk   []int32  
)
```

```
func isRoot(x int32) bool {  
    p := par[x]  
    return p == 0 || (ch[p][0] != x && ch[p][1] != x)  
}
```

```

func pushUp(x int32) {
    mx[x] = val[x]
    mxPos[x] = x
    if l := ch[x][0]; l != 0 && mx[l] > mx[x] {
        mx[x] = mx[l]
        mxPos[x] = mxPos[l]
    }
    if r := ch[x][1]; r != 0 && mx[r] > mx[x] {
        mx[x] = mx[r]
        mxPos[x] = mxPos[r]
    }
}

```

```

func pushDown(x int32) {
    if rv[x] {
        l, r := ch[x][0], ch[x][1]
        ch[x][0], ch[x][1] = r, l
        if l != 0 {
            rv[l] = !rv[l]
        }
        if r != 0 {
            rv[r] = !rv[r]
        }
        rv[x] = false
    }
}

```

```

func rotate(x int32) {
    y := par[x]
    z := par[y]
    var k int32
    if ch[y][1] == x {
        k = 1
    }
    if !isRoot(y) {
        if ch[z][0] == y {
            ch[z][0] = x
        } else {
            ch[z][1] = x
        }
    }
    par[x] = z
    ch[y][k] = ch[x][1-k]
    if ch[x][1-k] != 0 {
        par[ch[x][1-k]] = y
    }
    ch[x][1-k] = y
    par[y] = x
    pushUp(y)
    pushUp(x)
}

```

```

func splay(x int32) {
    stk = stk[:0]
    y := x
    stk = append(stk, y)
}

```

```

    for !isRoot(y) {
        y = par[y]
        stk = append(stk, y)
    }
    for i := len(stk) - 1; i >= 0; i-- {
        pushDown(stk[i])
    }
    for !isRoot(x) {
        y := par[x]
        z := par[y]
        if !isRoot(y) {
            if (ch[z][0] == y) == (ch[y][0] == x) {
                rotate(y)
            } else {
                rotate(x)
            }
        }
        rotate(x)
    }
}

func access(x int32) {
    var last int32 = 0
    for y := x; y != 0; y = par[y] {
        splay(y)
        ch[y][1] = last
        pushUp(y)
        last = y
    }
    splay(x)
}

func makeRoot(x int32) {
    access(x)
    rv[x] = !rv[x]
}

func link(x, y int32) {
    makeRoot(x)
    par[x] = y
}

func cut(x, y int32) {
    makeRoot(x)
    access(y)
    if ch[y][0] == x && ch[x][1] == 0 {
        ch[y][0] = 0
        par[x] = 0
        pushUp(y)
    }
}

func main() {
    reader := bufio.NewReaderSize(os.Stdin, 1<<20)
    n := readInt(reader)
    q := readInt(reader)

```

```

maxNodes := n + (n - 1) + q + 5
ch = make([][2]int32, maxNodes)
par = make([]int32, maxNodes)
rv = make([]bool, maxNodes)
val = make([]int64, maxNodes)
mx = make([]int64, maxNodes)
mxPos = make([]int32, maxNodes)
eu = make([]int32, maxNodes)
ev = make([]int32, maxNodes)
stk = make([]int32, 0, maxNodes)

for i := int32(1); i <= int32(n); i++ {
    val[i] = -1
    mx[i] = -1
    mxPos[i] = i
}

cnt := int32(n)
var total int64 = 0

for i := 0; i < n-1; i++ {
    a := int32(readInt(reader))
    b := int32(readInt(reader))
    c := int64(readInt(reader))
    cnt++
    e := cnt
    val[e], mx[e], mxPos[e] = c, c, e
    eu[e], ev[e] = a, b
    link(e, a)
    link(b, e)
    total += c
}

writer := bufio.NewWriterSize(os.Stdout, 1<<22)
defer writer.Flush()

for i := 0; i < q; i++ {
    u := int32(readInt(reader))
    v := int32(readInt(reader))
    w := int64(readInt(reader))

    makeRoot(u)
    access(v)
    maxW := mx[v]
    maxNode := mxPos[v]

    if w < maxW {
        total += w - maxW
        a2, b2 := eu[maxNode], ev[maxNode]
        cut(a2, maxNode)
        cut(maxNode, b2)

        cnt++
        e := cnt
        val[e], mx[e], mxPos[e] = w, w, e
    }
}

```

```

    eu[e], ev[e] = u, v
    link(e, u)
    link(v, e)
}

writer.WriteString(strconv.FormatInt(total, 10))
writer.WriteByte('\n')
}
}

func readInt(r *bufio.Reader) int {
    n := 0
    c, _ := r.ReadByte()
    for c == ' ' || c == '\n' || c == '\r' || c == '\t' {
        c, _ = r.ReadByte()
    }
    for c >= '0' && c <= '9' {
        n = n*10 + int(c-'0')
        c, _ = r.ReadByte()
    }
    return n
}

```